Context Inheritance: A Git-Native Architecture and Pre-Registered Study of Repository Context for AI Coding Agents

Pavel Kerbel*
pavel@getmetatron.com
Metatron Research
Tel Aviv, Israel

Milana Kerbel
milana@getmetatron.com
Metatron Research
Tel Aviv, Israel

Vitali Abramov
vitali@getmetatron.com
Metatron Research
Tel Aviv, Israel

Liat Abramov
liat@getmetatron.com
Metatron Research
Tel Aviv, Israel

Abstract

AI coding agents repeatedly rediscover project constraints, because repositories version code but not operating knowledge: the decisions taken, the alternatives rejected, the constraints found during execution. We introduce the *Repository Context Layer* (RCL)—the repository carries its own agent-facing context, versioned with the code, deterministically discoverable, consulted by contract, and writable by agents only under human review, in a minimal default format, `context.md`—and study, in a pre-registered experiment on SWE-bench Verified, when it helps. The organizing observation is a *capability-dependent author–reader pattern* we call *context inheritance*: repository context behaves like senior knowledge, and across the levels we observe its value flows toward weaker readers. A small local executor gains a large navigational benefit from context authored by a more capable model (+26 pp localization on topic-matched tasks, $p < 0.0001$) but nothing from context it wrote itself (+0.0 pp); with two author levels drawn from different model families, we report this as a pattern consistent with a capability gradient rather than an isolated causal variable. At the frontier the axes shift: an agent appears to gain from its own failure-derived lessons, but not from answer-derived ones (+14.6 pp resolve on constraint-sharing tasks; $p = 0.041$ uncorrected, 0.082 after Holm correction within its registered family, so we report the direction rather than a confirmed effect; −32% tokens per resolved task); *how* context is delivered matters (exploratorily, a sharded store the agent reads selectively outperformed a monolith at a third of the tokens at the local tier); and on a broad random sample a frontier agent sits at a resolve ceiling that no context improves. The design consequence is concrete: version repository context as a first-class artifact, author it with the strongest writer available, and expect the benefit in cheaper agents that read it.

CCS Concepts

• **Software and its engineering** → **Software creation and management**; • **Computing methodologies** → *Artificial intelligence*.

Keywords

repository context, agentic software development, context.md, capability gradient, context inheritance, agent memory, SWE-bench, pre-registered study

ACM Reference Format:
Pavel Kerbel, Milana Kerbel, Vitali Abramov, and Liat Abramov. 2026. Context Inheritance: A Git-Native Architecture and Pre-Registered Study of Repository Context for AI Coding Agents. In *Proceedings of International Workshop on Agentic AI for Next-Generation Software Development (AgenticDev 2026)*. ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Coding agents crossed a practical threshold in 2024–2026: given a task, a modern agent can plan, edit, run tests, and iterate over hours with little supervision. What has not kept pace is the substrate those agents work on. A repository exposes its code, but not the reasoning that produced the code. The result is a familiar failure mode: an agent that writes competent code while re-proposing the library the team removed, violating an unwritten layering rule, or re-discovering, at some cost, a constraint a previous session had already found.

A concrete case: an agent reaches for an ORM to simplify a query, unaware the team removed ORMs months earlier because query opacity had broken an offline repair path. The decision lives in commit history and human memory—nowhere the agent consults. It writes reasonable code that a reviewer must catch, or that ships and regresses a constraint the project had already settled.

Practitioners mitigate this with a patchwork: instruction files (`CLAUDE.md`, `AGENTS.md`, `.cursorrules`) inject standing guidance; IDE memories persist preferences per tool; retrieval-augmented generation (RAG) [2] recalls similar text; protocols such as MCP [3]

*Corresponding author. All authors are affiliated with Metatron Research (https://getmetatron.com); ORCIDs identify each author.

AgenticDev 2026, Munich, Germany

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

standardize access to external stores. Each helps; none yields a durable, reviewable, project-owned operating context that improves as work happens (Section 2).

This paper formalizes an alternative that requires no new infrastructure—the repository carries its own operating context, versioned by Git, gated by code review, and updated as a routine part of every change, a pattern we call the *Repository Context Layer* (RCL)—and then asks the question a practitioner actually faces: *given* such a layer, when does an agent benefit from it? The answer is not uniform, and its structure is the paper's central finding.

Context inheritance. We observe that the benefit an agent draws from repository context tracks the capability of whoever *wrote* the context relative to the agent that *reads* it. Context from a stronger author lifts a weaker reader decisively; a self-taught weak model gains nothing; a frontier reader already at its ceiling gains nothing generic delivery can add. Repository context behaves like senior engineering knowledge whose value flows toward cheaper agents that later read the repository. We call this pattern *context inheritance*. We observe two author levels drawn from different model families, so the experiment does not isolate capability as the sole causal variable; what it establishes is a consistent author–reader dependence, and that dependence reframes the practical questions of who should author a context layer, who benefits, and when it is worth the tokens.

Contributions.

(1) **The architecture.** A definition of the Repository Context Layer, its six required properties, a Git-native lifecycle with branch/merge/rollback semantics, a deterministic discovery specification, and a minimal file format (Sections 3–7).
(2) **Context inheritance: a capability-dependent author–reader pattern.** Holding a small executor fixed, a blind frontier-authored seed raises localization +26.1 pp ($p < 0.0001$) on topic-matched tasks while the executor's own lessons move it +0.0 pp ($p = 1.0$). What transfers appears to depend on the reader: directionally, a frontier agent draws on failure-derived lessons rather than the answer-derived ones that serve the weak tier—a suggestive contrast that does not clear correction for multiple comparisons (Sections 10.2–10.3).
(3) **When inheritance pays, and what it costs.** Delivery matters—a sharded store the agent reads selectively outperformed a monolith at a third of the context tokens, an exploratory contrast (Section 10.4)—but inheritance has a ceiling: on a broad random sample a frontier agent resolves 91% of well-specified bugs unaided and no delivery improves on it (Section 10.5). Where it does pay it is self-financing, at −32% tokens per resolved task (Section 10.6).

We keep four concerns separate, because they can be adopted independently: the *architecture* (what the layer is), the *discovery specification* (how agents find it), the *file format* (the default representation), and *implementations* (tools that automate it). A team can conform with nothing but a Markdown file and a code-review habit.

2 Background

We survey the mechanisms teams currently use to convey project knowledge to agents, against five properties the problem demands: persistence across sessions, co-versioning with the code, agent writability, human review of changes, and consultation by contract (the agent is required to read it, rather than hoping retrieval surfaces it).

*README*s *and documentation.* Durable and versioned, but usage-oriented: they explain how to build and run, rarely why the architecture is shaped as it is, and almost never what was rejected. Documentation also drifts, because nothing in the workflow forces an update when a decision is reversed.

Architecture decision records. ADRs [4] are the closest ancestor of this work. They capture context, decision, and consequences, and they live in the repository. But ADRs are write-once records addressed to future humans. There is no contract obliging an agent to read them, no convention for machine consumption, and no channel for an agent to contribute what execution taught it. In practice ADR directories are archives, not operating context.

Wikis and design documents. Detached from the repository, unversioned with the code, and notorious for rot. A wiki page cannot branch with an experiment or roll back with a revert.

RAG over project artifacts. Retrieval by embedding similarity [2] answers "what text resembles this query," which is the wrong question for constraints. A prohibition matters most precisely when nothing in the current prompt resembles it; an agent adding a dependency will not emit a query similar to the paragraph explaining why dependencies are frozen. RAG also adds infrastructure, and its store is not reviewable by diff.

Agent memory systems. A rich research line gives agents persistent memory: Reflexion [6] stores verbal self-reflections on failures and improves within a task series; Voyager [7] accumulates an ever-growing library of validated skills; MemGPT [8] pages long-term memory in and out of the context window; generative agents [9] maintain retrieval-scored memory streams. These systems demonstrate that experience-derived memory improves agent performance—the premise our evaluation revisits in a software-engineering setting—but their stores are agent- or platform-owned: unversioned with any codebase, invisible to code review, and not shared across heterogeneous tools working on the same project. RCL relocates exactly this memory into the repository's own trust machinery: the store branches, merges, and reverts with the code, and every write passes human review. Our results add a boundary condition this literature has not measured: the value of self-derived memory depends sharply on the capability of the agent writing it (Section 10).

IDE and platform memories. Convenient and automatic, but scoped to one tool and often one machine, invisible to teammates, and unreviewed. Two agents working the same repository through different tools share nothing.

Agent instruction files. `CLAUDE.md`, `AGENTS.md`, `.cursorrules`, and similar files are versioned, reviewed, and consulted by contract, which is why they spread quickly. Their limitation is directionality: they carry orders from the human downward, with no defined way to absorb what the agent itself learns. They are static configuration, not evolving context.

Table 1: Project-knowledge mechanisms against the properties persistent agent context requires.

Mechanism	Versioned w/ code	Agent-writable	Human-reviewed	Consulted by contract
README / docs	☒	—	☒	—
ADRs	☒	—	☒	—
Wiki / design docs	—	—	—	—
RAG store	—	partial	—	—
IDE memories	—	☒	—	—
Agent memory systems	—	☒	—	partial
Instruction files	☒	—	☒	☒
Context layer	☒	☒	☒	☒

Protocols. MCP [3] and similar protocols standardize transport between agents and external systems. Transport is orthogonal to persistence: a protocol can serve a context layer, but does not itself provide one.

The pattern in Table 1 is the thesis of this section: each mechanism solves a slice, and the missing artifact is the one with all four properties at once.

3 The Repository Context Layer

A Repository Context Layer is a version-controlled, human-readable context store that lives alongside the codebase and serves as the authoritative operating context for AI agents working in it. The distinction from adjacent artifacts is one of question: code captures what a system does; documentation explains how to use it; the context layer records *why* future changes should or should not be made—the intent behind the design and the constraints that must survive it.

3.1 Required properties

An implementation conforms to the architecture if it satisfies six properties. **P1 Co-versioned:** the context is stored in the repository and follows its branching model. **P2 Human-readable:** the representation is plain text a maintainer can read and edit without tooling. **P3 Deterministically discoverable:** an agent locates the context by a fixed rule, not by search (Section 5). **P4 Bidirectional:** agents both consume the context and propose additions to it. **P5 Review-gated:** no change to the context takes effect without passing the project's normal review process. **P6 Tool-independent:** participation requires only the ability to read and write files.

3.2 Goals and non-goals

The layer aims to hold the *operating* knowledge of a project: intent, binding constraints with their reasons, and validated lessons from execution. It is deliberately not a knowledge base of the code itself (the code is present and searchable), not a replacement for retrieval over large corpora, not a record of conversations, and not a configuration system. A useful test: an entry belongs in the layer if a competent new engineer would need to be told it, and would not discover it by reading the code.

3.3 Relationship to Git

The architecture makes one load-bearing choice: it defines conventions over plain files in Git rather than a new store. This buys provenance (`git blame` answers who believed what, and when), review (the pull request is the approval mechanism), branching and rollback (Section 4), distributed synchronization, and audit, all without new infrastructure. The cost is Git's limitations, notably textual rather than semantic merging; Section 7 discusses the consequences.

3.4 Four layers

We separate the pattern into independently adoptable layers: **L1**, the architecture of this section; **L2**, the discovery specification of Section 5; **L3**, the `context.md` file format of Section 6; and **L4**, implementations that automate consultation, capture, and retrieval. A team may adopt L1–L3 with no tooling at all; a vendor may implement L1–L2 with a different representation than L3. Conformance claims should name the layers they cover.

4 The Context Lifecycle

The lifecycle is the behavioral half of the architecture: it defines when the context is read and when it is written: *consult* → *plan* → *execute* → *observe* → *update* → *review* → *commit*.

Consult. Before planning, the agent reads the context in full. Constraints bind the plan; intent breaks ties among admissible designs. *Plan / Execute.* Ordinary agent operation. *Observe.* On completion, the agent asks what the work taught that a future session would otherwise re-learn. Most observations are discarded; durability is the filter. *Update.* Surviving observations are appended to the context's ledger. *Review.* The context diff rides in the same change set as the code, and a human approves both together. *Commit.* The repository's knowledge and behavior advance atomically.

Four invariants summarize the contract. **I1:** consultation precedes planning. **I2:** ledger writes are appends. **I3:** no context change takes effect without review. **I4:** context changes travel with the code changes that motivated them.

Branch behavior. Because the context is a file, it branches when the code branches. A learning made on an experimental branch is visible only there, which is correct: it may be true only there. Deleting an abandoned branch deletes beliefs that were never validated. *Merge behavior.* Merging a branch merges its context. Appends to a ledger from different branches usually touch different lines and merge cleanly; a union merge strategy makes this explicit. Conflicting edits to a constraint are semantic conflicts, and the architecture deliberately routes them to a human: two branches that learned incompatible things have surfaced a real disagreement. *Rollback.* Reverting a commit reverts the beliefs recorded in it. This is a feature: if the change that motivated a learning is withdrawn, the learning's justification is withdrawn with it.

5 Context Discovery Specification

Discovery must be deterministic (P3): a fixed path is an API, and an agent must not depend on search or inference to find its own operating contract. The key words MUST, SHOULD, and MAY follow RFC 2119 [5]. An agent MUST check `.repo/context.md`, then `context.md` at the repository root, and use the first file found.

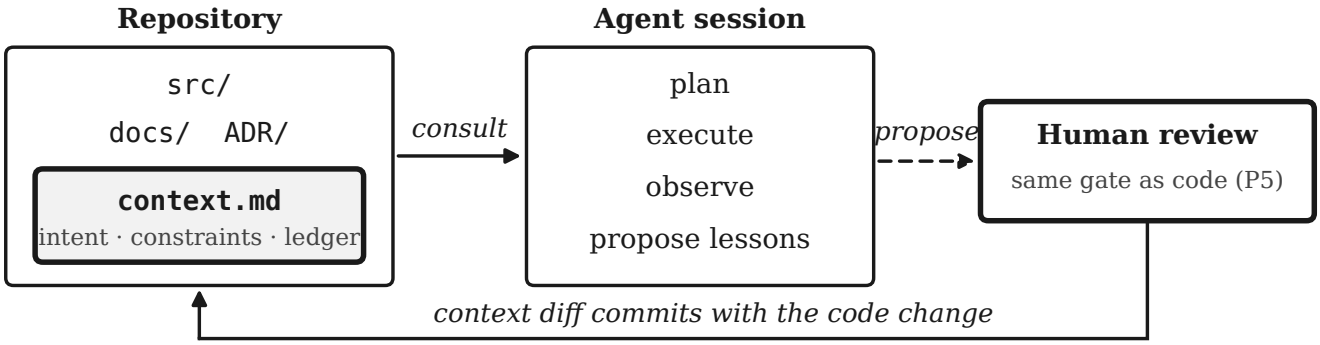

Figure 1: The Repository Context Layer is added alongside a repository's existing artifacts. An agent consults it before planning and proposes updates after executing; every update travels through the same review as the code that motivated it.

The paths `.repo/context/` and `context/` (repository root) are RESERVED for sharding of large contexts. Agents encountering either before they support sharding MUST still honor the first rule; a repository that shards SHOULD keep a root context file as the discoverable index into the shards. Agents MUST ignore sections they do not understand and MUST NOT fail on additional files or metadata.

6 The Context File Format

The default representation (L3) is a single CommonMark file with a top-level `# Repository Context` heading and three required sections, in any order. *Intent* states what the project is and the design philosophy subordinate decisions must serve; it is the tiebreaker among admissible designs. *Constraints* are the binding rules; each entry SHOULD carry its reason, including what was rejected and why. Constraints are edited, not appended. *Evolved Context* is an append-only ledger of dated entries recording what agents and humans learned during execution. Entries are never rewritten; corrections are recorded as superseding entries. Entries that prove durable are *promoted*: moved into Constraints in a reviewed change.

Existing projects rarely start empty. A practical seeding procedure: distill each ADR's decision and consequences into one Constraints entry; move durable guidance out of instruction files; and start the ledger empty. Seeding is a single reviewed commit.

7 Design Discussion

Five design choices attract consistent pushback; we record the reasoning.

One file or many. One file is the default because the context is a working set, not an archive: its purpose is to be read, in full, at the start of every session. When a context genuinely outgrows one file, the reserved directories admit sharding by topic, with the root file retained as the discoverable index. The failure mode to resist is archival creep: a context file that tries to be documentation stops being read.

Scope of append-only. Append-only applies to the ledger, not the layer. The ledger is an evidence log; rewriting evidence destroys the provenance that makes it trustworthy. Constraints, by contrast, are current law and are edited in place, with Git history preserving every prior state.

Merge conflicts. Textual conflicts concentrate in the ledger, where concurrent appends collide at the file's tail. Declaring a union merge for the context file resolves concurrent appends automatically while leaving edits to shared lines for humans. The architecture automates the merges that cannot be wrong and refuses to automate the ones that can.

Hierarchical context. Monorepos want scoped context. The v0.x position is to standardize the root and defer hierarchy, reserving nearest-ancestor discovery as the intended extension. Hierarchy multiplies the spec surface, and the pattern should earn complexity with adoption evidence first.

Agent write authority and the threat model. An agent may propose anything and enact nothing. Every context change lands through the same review gate as code (I3), so the ceiling on agent authority equals the rigor of the project's review process. The gate is also the security boundary: a context layer is standing instruction to future agents, making a poisoned entry persistent prompt injection. Three properties contain this: review-gating (P5) means hostile entries must pass a human; provenance (P1) means every entry has an author, a commit, and a motivating change; human-readability (P2) denies attackers obscurity. Reviewers SHOULD treat context diffs with the scrutiny of code diffs, and conformant tooling MUST refuse to promote files that do not explicitly declare themselves context documents. The gate must also scale: an agent attempting dozens of tasks a day would exhaust reviewer attention if every raw lesson demanded fresh scrutiny. Automated pre-screening therefore belongs in front of the human—format and size linting, classifiers that flag privilege-expanding or outward-pointing instructions, and mechanical promotion rules such as the rubric our evaluation uses—so the reviewer sees few, well-formed, pre-vetted diffs rather than a stream of candidates. In our evaluation such a rubric screened every candidate lesson with no human in the loop, rejecting roughly one in ten before review—chiefly

entries too narrow to be repository-general (naming a specific instance or line number) and, less often, over-length entries failing the size lint.

8 What the Layer Is Not

It is not RAG: consultation is by contract, not similarity. It is not prompt engineering: instruction files configure behavior top-down and are static between human edits, while the layer evolves through execution. It is not documentation: docs address human onboarding at human timescales; the layer is machine-facing operating state. And it is not chat memory: platform memories attach to a user or a tool, while repository context attaches to the artifact all users and tools share. The distinctions matter because each framing suggests the wrong lifecycle: archives are allowed to go stale, and this must not.

9 Reference Implementation

A reference implementation, Metatron [12], implements all four layers (L1–L4) for teams that want the lifecycle automated: project knowledge as individual Markdown files in an open knowledge format, one decision per file with structured front matter, sharded under a root-level `context/` directory, with a scaffolded root `context.md` whose Constraints point into the shards—so discovery works for any conformant agent with no tool-specific knowledge. Candidate entries flow through an explicit review step before becoming canonical. The implementation exists to reduce friction, not to be required: a team using plain `context.md` with disciplined review conforms to the same architecture.

Current agents need a one-time bootstrap telling them the contract exists; the natural place is the instruction file they already read (`CLAUDE.md`, `AGENTS.md`, `.cursorrules`, or a system-prompt sentence). This yields a clean division of labor: the instruction file tells the agent that the context layer exists; the context layer holds what the agent needs to know. Because every integration reads the same artifact, a repository worked on by three different tools accumulates one shared context rather than three private memories.

10 Empirical Evaluation

We evaluate the architecture's central behavioral claims—that repository-carried context measurably changes agent behavior, and that its benefit is capability-dependent—in a pre-registered experiment on SWE-bench Verified [10, 11]. The protocol (hypotheses, conditions, metrics, analysis plan, power simulation) was frozen and externally timestamped at a git tag before any confirmatory run; pilots are excluded. All materials—the frozen protocol, blind-authoring audit, transcripts, promotion logs, and containerized evaluation verdicts—are released [13].

10.1 Design

Transfer pairs. SWE-bench Verified supplies real historical bugs, each with a hidden grading suite and the maintainers' fix (the *gold patch*). We identified *constraint groups*—instances within one repository whose fixes depend on the same convention (e.g., xarray's `keep_attrs` propagation; Django's `deconstruct()` fidelity rules)—and froze ordered transfer pairs (A, B) before any run: B is held out, and the treatment's only advantage on B is context derived from working on A. This constraint-sharing regime is about 8% of the benchmark—the setting the architecture is designed for—and we also evaluate a broad random sample (Section 10.5).

Conditions. Executors: a frontier model (Claude Opus 4.8) and a small local model (Gemma 3, 8B, quantized), in an identical minimal scaffold; every episode is a fresh session in a freshly cloned checkout, no state carried across episodes. Context authors: none; the executor itself (self-authored, via the lifecycle); a more capable model given only a years-old, history-stripped checkout and constraint-group names, never a problem statement, patch, or test (a blind seed); or a labeled oracle bound shown the gold patch (never pooled). **No context was produced by an automatic extraction pipeline**: all authored context is blind- or lifecycle-derived by design, isolating the context-author condition while holding the executor and delivery setup fixed, and limiting solution leakage (audited in Section 10.7). Delivery: injected into the prompt, a monolithic `context.md`, or a sharded store read selectively under a consultation contract. Metrics: consultation, gold-file localization, resolve (official containers), and tokens. Paired comparisons use sign-flip permutation tests (10k draws, fixed seed); families are Holm-corrected.

Table 2 states the full matrix once; every result section below is one row of it. Two task sets recur and should not be conflated: the *topic-matched* set (the 36 frozen transfer pairs, where held-out task B shares a constraint with A) and the *broad* sets (a 267-instance pool and an 88-instance random sample, neither selected for shared constraints). The architecture targets the first; the second bounds what it is worth elsewhere.

Table 2: The complete experimental matrix. Each results section varies one axis and holds the rest fixed. "Frontier seed" was authored from a history-stripped checkout and constraint-group names only; "oracle" arms were shown the gold patch of the *source* task A and are never pooled with confirmatory arms. *Matched* task sets are the frozen transfer pairs, in which the held-out task shares a constraint with the source task; *broad* and *random* sets are not selected for shared constraints.

§	Executor, metric	What varies	Task set (ep./arm)
10.2	8B, localization	author: none, self, frontier seed, oracle	36 matched (180)
10.3	frontier, resolve	author: none, self (failure), oracle (answer)	16 matched (48)
10.4	8B, localization	delivery: none, inject, file, shard	267 broad (801)
10.5	frontier, resolve	delivery: none, inject, file, shard	88 random (88)
10.6	frontier, tokens	author: none, self (failure), oracle	16 matched (48)

10.2 Context from a stronger author benefits the local executor

Holding the executor fixed at 8B (36 pairs × 5 repetitions = 180 held-out episodes per arm), the held-out outcome varies only with who authored the context (Figure 2). A blind frontier-authored seed raises gold-file localization from 21.1% to 47.2%—+26.1 pp, $p <$

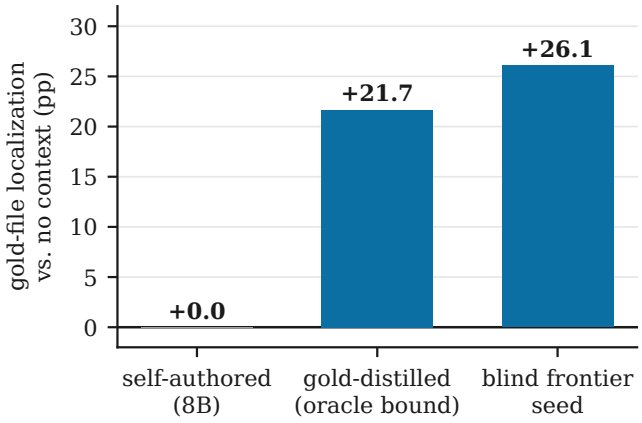

Figure 2: Paired improvement in the 8B executor's gold-file localization over its no-context control, by context author (36 pairs × 5 repetitions). Both capable-author conditions approximately doubled localization here; the executor's self-authored context sat on the no-context floor. Resolve deltas are not significant at this tier.

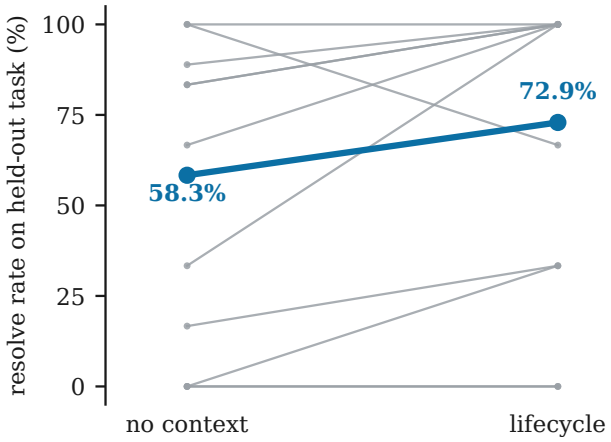

Figure 3: Frontier executor resolve rate on held-out tasks, without and with the lifecycle. Gray lines: the 11 constraint groups (7 improve, 3 are flat, 1 declines); the bold line is the overall episode-level mean. The +14.6 pp difference is directional: $p = 0.041$ uncorrected, 0.082 after Holm correction within its registered family.

0.0001—and a gold-distilled bound to 42.8% (+21.7 pp, $p < 0.0001$); the two capable-author tiers are statistically indistinguishable. The executor's *own* promoted lessons—162 across 60 pair-repetitions—move localization by +0.0 pp ($p = 1.0$): exactly the no-context control. In this experiment, then, context from either capable author approximately doubled the 8B executor's localization relative to no context, while its self-authored lessons produced no measurable localization improvement. We observe two author levels only, so we cannot distinguish a threshold from a steep dose curve; author and executor also come from different model families, so author capability and family characteristics are not separated here. A within-family author-size sweep would settle both; we did not run one. The 8B model used frontier-written context effectively, but its own lessons did not improve localization under this setup: it recorded tooling tactics rather than the repository constraints the frontier author captured under the identical prompt. At this tier the movement is in localization, not resolve (all arms 5–8%; pooled oracle +2.2 pp, $p = 0.22$).

10.3 At the frontier, the transfer pattern differs

Running the full lifecycle with the frontier executor on 16 constraint-sharing pairs (× 3 repetitions, 48 held-out episodes/arm), the agent's own failure-derived lessons—extracted from working task *A*, never seeing a gold patch or hidden test—raise held-out resolve on *B* from 58.3% to **72.9%** (+14.6 pp; localization 91.7 vs. 81.2%; of the 11 constraint groups 7 improve, 3 are flat, and 1 declines; Figure 3). These are uncorrected paired permutation *p*-values (0.041 and 0.064); Holm correction across the two contrasts in this registered family puts both at 0.082, so the effect does not clear the pre-registered bar and we report its direction rather than a confirmed difference. H2 (treatment over control on held-out *B*) was pre-registered; the *inversion* reading below—that what transfers switches from answers to failures as the reader gets stronger—was not a registered hypothesis and is post hoc. Yet lessons distilled from the *gold patch* transfer nothing at this tier (60.4% resolve, indistinguishable from no context). The observed pattern is directionally opposite to the weak tier, where gold-distilled context was the strongest author: answer-derived lessons encode solutions; failure-derived lessons encode traps and process. A weak reader needs to be told where things are; a strong reader already localizes and needs to be told where it will go wrong. What is inherited depends on the reader.

10.4 Delivery shapes inheritance

Holding content fixed (the blind seed) and varying only *how* it is delivered, we ran four conditions with the local executor on a frozen 267-instance pool (3 repetitions, 801 episodes/arm): no context (B), injected (E), a monolithic file (FILE), and a sharded store the agent reads selectively (SHARD). The procedural consultation contract binds at scale—FILE and SHARD are consulted in 97% of episodes, confirming the agent reliably reads the layer when instructed. The delivery mechanisms then separate (Table 3). SHARD is best on localization (53.4%) and, decisively, cheapest: it pays ~3× fewer context tokens than the monolith (305 vs. 902 per episode, $p = 0.0002$) because the agent reads the index plus only the relevant shard(s), while FILE forces the whole file through the transcript. The monolith is numerically *below* the no-context baseline on localization (46.8 vs. 48.6%) while paying the most tokens—plausibly crowding task reasoning in a bounded window. Under the registered family (each mechanism against no-context and against injection, Holm corrected), no single contrast clears the bar—SHARD—B is +4.9 pp, Holm $p = 0.058$, a hair short—but the sharpest separation in the data is between the two file mechanisms themselves. Exploratorily, selective sharding outperformed the monolithic file by +6.6 pp ($p = 0.0007$) while using roughly one third of the context tokens. These exploratory results suggest that selection can outperform

Table 3: Local executor (8B), delivery of identical content. 801 episodes/arm. Context tokens are the payload read into the transcript.

delivery	localization	consulted	ctx tokens /episode	resolve
none (B)	48.6%	—	0	5.6%
injected (E)	51.2%	—	930	8.6%
file (FILE)	46.8%	97%	902	7.0%
sharded (SHARD)	**53.4%**	97%	**305**	6.7%

volume: making the agent *choose* what to read both helped and cost less here. If that reading holds, it extends the gradient's lesson from *who authored* the context to *how much* of it is placed before the reader.

10.5 Inheritance has a ceiling

The results above hold in the constraint-sharing regime the layer is built for. Outside it, the picture changes. We ran all four delivery conditions with the frontier executor on an 88-instance stratified subset (one repetition; Figure 4). The no-context baseline already resolves **90.9%** of these well-specified, isolated bugs: the frontier model does not need repository context to fix bugs it can already localize, and there is no headroom for context to add resolution. No delivery beats the baseline; the small differences among B/E/FILE (± a few instances on $n = 88$) are within noise. The contract still binds—FILE and SHARD read the context in 100% of episodes—so the null is not a failure to read; it is that reading generic context on a random bug has nothing to contribute. Here the selective store's economics also *invert*: a capable model turns "read only what you need" into thorough exploration, so SHARD spends 38.4K prompt tokens against FILE's 24.0K and costs ~50% more per resolved task—the opposite of the local tier, where SHARD was cheapest. Inheritance requires a reader below its ceiling as well as an author above it; a frontier model on well-specified bugs violates the first condition, and generic context becomes a cost, not a benefit.

10.6 Where it pays, inheritance is self-financing

In the constraint-sharing regime, the frontier lifecycle spends 55.7K tokens per resolved task against 81.8K with no context (−32%; Figure 5)—and the decomposition is telling: episodes ending in a fix cost the same in both arms (≈33.6K); the entire saving comes from fewer doomed explorations. The oracle arm saves tokens identically without converting them into resolves: context of either kind shortened wandering here, but only the failure-derived condition also shows the directional resolve improvement reported in Section 10.3. The local executor shows the opposite sign (+20% tokens per episode with the seed): for a strong reader in the right regime context is self-financing; for a weak one it is a paid navigational upgrade. The whole experiment—roughly 2,100 episodes and 850 containerized evaluations—consumed a few hundred dollars of frontier spend, consistent with the economic premise: expensive knowledge, written once, read cheaply forever.

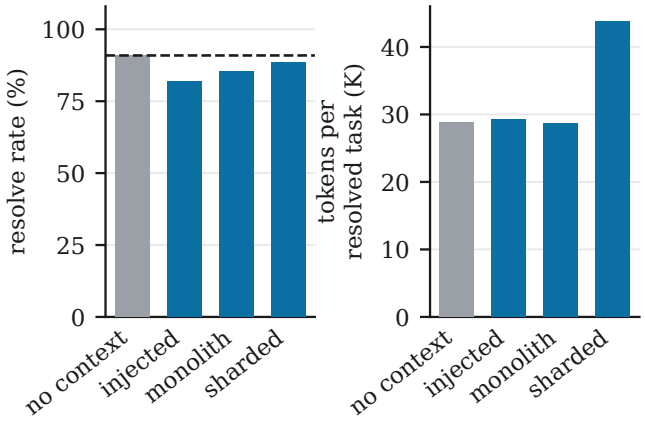

Figure 4: Frontier executor on a broad 88-instance random sample: resolve rate by delivery. No arm exceeds the no-context ceiling (dashed); the sharded store recovers most but at the highest per-resolved cost. Costs are notional, at list prices, excluding prompt caching.

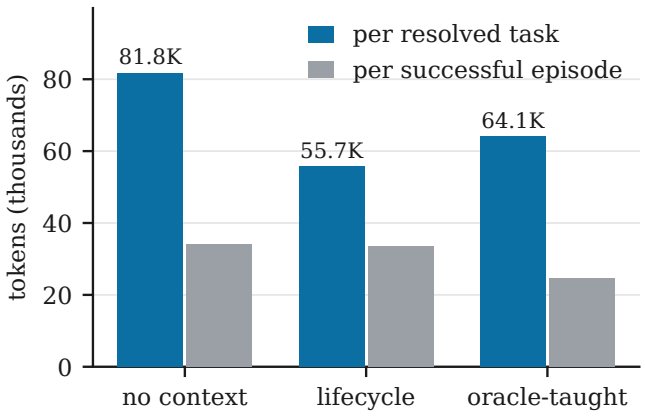

Figure 5: Tokens per resolved task (frontier executor, constraint-sharing regime). Successful episodes cost the same in both arms; the saving comes from fewer failed explorations. The oracle-taught variant spends like the lifecycle arm without matching its resolve rate.

10.7 Threats to validity

Benchmark scope. SWE-bench Verified supplies well-specified, isolated bug fixes graded by hidden tests—a task type where localization is half the battle and a frontier model already localizes almost unaided, which is precisely why the ceiling (Section 10.5) appears. It cannot measure a context layer's central real-world value—adherence to conventions and avoidance of rejected approaches—where "correct" is not "tests pass." The frontier ceiling is thus a statement about this task type, not about repository context in general: it is in long-lived repositories carrying implicit architectural constraints, not in isolated well-specified fixes, that a frontier reader would retain headroom for context to lift. A benchmark scoring convention-adherence is the natural next

instrument. *Extraction untested.* All authored context here is blind- or lifecycle-derived, deliberately, to vary the author-capability contrast and limit leakage; whether an automatic pipeline can *produce* context of comparable value is a separate question this design brackets, and the clearest follow-up. *Statistical scope.* The gradient (180 episodes/arm) and delivery matrix (801/arm) are the robust core; the frontier resolve effect ($n = 16 \times 3$) and broad-sample matrix ($n = 88 \times 1$) have wider intervals and are reported with their caveats. Potential executor contamination affects both arms of within-model comparisons symmetrically and would tend to compress deltas toward zero. Potential contamination of the seed *author* is not symmetric—it could favor the treatment—and is considered separately in the leakage audit below. A local-tier resolve improvement marginal in the initial pair set ($p = 0.066$) did not survive the pre-planned expansion (pooled $p = 0.22$) and is not claimed; the frontier failure-vs-answers contrast is marginal ($p = 0.067$) and reported as directional.

Blind-seed leakage audit. The seed author never sees a problem statement, patch, or test, but it does receive the repository and the constraint-group names—and those names point at the convention under test, while these repositories are almost certainly in the frontier author's pretraining. A seed that partially distilled the answer would bias the treatment rather than cancel, so we audited the seeds against the held-out gold patches (Table 4). The seed names a file the fix edits for 83% of held-out instances against 32% for same-repo instances it was never directed at, and a symbol the fix touches 56% versus 4% of the time. This is strong evidence of topic and localization conditioning: the seed reliably knows *where* the relevant machinery lives, which is unsurprising given that an experimenter chose the constraint groups it was pointed at. Surface overlap with the fix itself is much weaker (identifier Jaccard 0.019 vs. 0.013 on the same seed text, which holds length fixed), which is some evidence against literal reproduction of patch content. Two limits should be read with these numbers. First, the audit measures textual overlap, so it cannot rule out partial solution knowledge mediated by pretraining and expressed in different words. Second, it is an observational audit, not a manipulation. Taken together the results are *consistent with* a primarily navigational effect, and the delivery experiment points the same way—the identical seed is worth +26.1 pp on topic-matched pairs but +4.9 pp on the broad 267-instance pool (Section 10.4)—but they do not settle the contamination question. We therefore treat the +26.1 pp figure as explicitly conditional on experimenter-selected topic match, and +4.9 pp as the more task-agnostic estimate. Analysis code and per-instance scores are released with the artifact [13].

Specification scope versus evidence scope. The normative half of this paper is broader than the experiment. The study exercises discovery (Section 5), the consultation contract (consulted in 97–100% of episodes), the lesson lifecycle (Section 4), and the file format. It does *not* measure the review gate's usefulness, branch/merge/rollback behavior under concurrent edits, or containment of a poisoned entry; those clauses are proposed convention, justified by analogy to code review rather than by evidence here, and should be read as such.

Table 4: Overlap between the blind seed and the gold patch of the instance the executor was held out on, against the same seed text scored on same-repo instances outside the study set. Holding the seed fixed across both columns controls for length.

Seed overlap with the gold patch	held-out B ($n = 36$)	non-study ($n = 257$)
names a file the fix edits	83.3%	32.3%
names a symbol the fix touches	55.6%	3.9%
identifier Jaccard with the patch	0.019	0.013

11 Discussion and Future Work

The results carry three practical consequences. Authorship should go to the most capable writer available—a human expert, a frontier model, or distillation from authoritative fixes—rather than to the working agents themselves, whose self-authored context did not help them here. The beneficiaries are those cheaper agents: one authoring pass, paid once, propagates navigational knowledge to every agent that later reads the repository at zero marginal inference cost. And the layer earns its tokens only where the reader sits below its ceiling and the context matches the task at hand—weaker models broadly, stronger models on work that is ambiguous, convention-laden, or re-hits a known constraint.

Future work. Several directions follow. *Validation:* conformance linting for the format and, more usefully, for the lifecycle—CI integrations (e.g., a pull-request action producing semantic context-diff summaries) that flag substantive code changes landing with no context diff over long periods. *Semantic merging:* LLM-assisted reconciliation of contradictory ledger entries, proposed to a human rather than applied. *Summarization and budgets:* automatic distillation of aging ledgers so the working set stays small enough to be read in full. *Hierarchy:* the nearest-ancestor extension for monorepos that want scoped context. *Cross-repository inheritance:* organization-wide constraint layers that member repositories extend—the capability gradient suggests a single well-authored parent could lift many child repositories at once. *Beyond the benchmark:* SWE-bench measures isolated bug-fix correctness; a benchmark that scores convention-adherence and the avoidance of rejected approaches would probe the value the frontier ceiling hides. And—most pointedly, given the extraction result this study deliberately brackets—measuring whether an automatic extraction pipeline can author context of frontier-blind quality, by varying the context's author (blind-frontier vs. machine-extracted vs. human-curated) with delivery and executor held fixed.

12 Conclusion

Software repositories version their code, tests, documentation, and infrastructure; this paper argues, and measures, that they should also version their operating context. The architecture asks for no new infrastructure, reuses the trust machinery of Git and code review, and degrades to a single Markdown file. Whether it helps appears to be less a property of the layer than of the relation between its author and its reader—the same seed is transformative for a small executor on matched work and inert for a frontier model

on a random bug. Within the levels we could observe, the practical reading is to author repository context with the strongest writer available and to expect the benefit in cheaper agents that read it.

Conflicts of Interest

All authors are affiliated with Metatron Research, whose open-source implementation of the repository-context pattern (Metatron) motivates this study. The experiment mitigates this competing interest by pre-registration before data collection, a scaffold-uniform minimal harness rather than the product itself, context that is blind- or lifecycle-authored rather than produced by that implementation, and fully public data and analysis code.

AI-Assisted Preparation Statement

Consistent with COPE guidance, the authors disclose that large language models (Anthropic Claude) assisted with manuscript drafting, analysis scripting, and experiment orchestration under continuous author direction and review. The authors take full responsibility for the final text and all reported results.

Data Availability

Every empirical claim is reproducible from the released replication package [13], archived at **[archival DOI pending deposit]**: the pre-registration frozen at a verifiable git tag before any confirmatory run; the seed-authoring audit record (`seeds/AUDIT.md`), which documents the blind-authoring procedure, the inputs given to each author and the prohibitions imposed; the leakage audit and its analysis code; the complete harness; every episode's predictions, promotion decisions, and containerized evaluation verdict; and the analysis scripts. The development repository is https://github.com/kerbelp/context-md.

References

[1] P. Kerbel. "The Repository Context Layer: The Missing Layer of Agentic Software Development," manifesto and specification repository, 2026. https://github.com/kerbelp/repository-context-layer.

[2] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS 33*.

[3] Anthropic. 2024. Introducing the Model Context Protocol. https://modelcontextprotocol.io.

[4] Michael Nygard. 2011. Documenting Architecture Decisions.

[5] Scott Bradner. 1997. Key Words for Use in RFCs to Indicate Requirement Levels. IETF RFC 2119.

[6] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *NeurIPS 36*.

[7] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, et al. 2024. Voyager: An Open-Ended Embodied Agent with Large Language Models. *TMLR*. arXiv:2305.16291.

[8] Charles Packer, Sarah Wooders, Kevin Lin, et al. 2023. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560.

[9] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents. In *UIST '23*. ACM.

[10] C. E. Jimenez et al. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *ICLR 2024*.

[11] OpenAI. 2024. Introducing SWE-bench Verified: a human-validated subset of SWE-bench. https://openai.com/index/introducing-swe-bench-verified/.

[12] P. Kerbel. "Metatron: a files-first repository context layer for AI coding agents," software, v0.12.0, 2026. https://github.com/kerbelp/metatron.

[13] P. Kerbel, M. Kerbel, V. Abramov, and L. Abramov. "RCL efficacy study: pre-registered protocol and replication artifacts," 2026. **[archival DOI pending deposit]** (development repository: https://github.com/kerbelp/context-md).